MERIDIAN MOBILITY SYSTEMS

Intelligent Transportation Systems · Data & AI Division

PRODUCT REQUIREMENTS DOCUMENT

FlowCast

Smart Traffic Flow Prediction System

An end-to-end AI system for forecasting urban traffic volume, congestion, travel time and accident risk across an arterial road network.

Document type	Product Requirements Document (PRD)
Programme	Capstone / Final-Year AI Engineering Project
Delivery model	Individual contributor · 4-week sprint
Version	1.0
Status	Approved for build
Owner	Data & AI Division, Meridian Mobility Systems
Classification	Internal — Engineering

Contents

1. Project Overview 4
2. Business Problem 4
3. Objectives 5
3.1 Product objectives 5
3.2 Engineering objectives (learning outcomes) 5
3.3 Success criteria 5
4. Functional Requirements 5
4.1 Data acquisition & preparation 6
4.2 Modelling & prediction 6
4.3 Analytics & dashboard 6
5. Non-Functional Requirements 7
6. User Roles 7
7. System Modules 7
8. Dataset Specification 8
8.1 Source tables 8
8.2 Merged schema (analysis-ready record) 8
8.3 Known data-quality issues to expect 9
9. Data Cleaning Pipeline 10
9.1 Ordered stages 10
9.2 Feature engineering detail 10
10. Mathematics Used 11
10.1 Concept-to-application map 11
10.2 Evaluation mathematics 11
11. Machine Learning Workflow 12
11.1 Targets and framing 12
11.2 Models to build and compare 12
11.3 Workflow steps 12
11.4 Model-selection justification 13
12. Deep Learning Workflow 13
12.1 Model choice 13
12.2 Training procedure 13
12.3 Overfitting prevention 13
12.4 Benchmark against classical models 14
13. Model Evaluation Strategy 14
13.1 Validation design 14
13.2 Metrics by task 14

13.3 Error analysis 14
13.4 Model governance 14
14. Dashboard Design 14
14.1 Required views 15
14.2 Supporting modules 15
14.3 Design principles 15
15. Technology Stack 15
16. Folder Structure 16
17. Development Phases 16
Week 1 — Data engineering & EDA 16
Week 2 — Classical machine learning 17
Week 3 — Deep learning 17
Week 4 — Dashboard, reporting & packaging 17
17.1 Milestone summary 17
18. Expected Deliverables 17
19. Future Enhancements 18
Tools Required 18
Hardware 18
Core software & libraries 18
Skills the engineer must bring 18

1. Project Overview

Meridian Mobility Systems (MMS) operates traffic-intelligence platforms for metropolitan road authorities. FlowCast is the company's short-horizon traffic prediction product. It ingests road-sensor telemetry, vehicle counts, weather feeds and calendar signals for a defined arterial road network, and produces forward-looking forecasts of traffic volume, congestion level, expected travel time and accident risk for each road segment and time window.

This PRD scopes a single-engineer build of FlowCast to be delivered over a four-week sprint. The engineer is responsible for the complete pipeline: raw-data acquisition, cleaning and validation, exploratory analysis, feature engineering, a family of classical machine-learning models, a deep-learning sequence model implemented and trained from scratch, a rigorous evaluation protocol, and an interactive analytics dashboard that surfaces predictions and insights to operations staff.

The build is scoped for the Northline Corridor programme — a 25-segment arterial network monitored by inductive-loop and radar sensors, with 30-minute aggregation. FlowCast is a decision-support layer; it does not control signals directly. Its outputs feed the corridor operations team, who use forecasts to plan signal-timing adjustments, incident response and public travel advisories.

> **Scope statement**
> FlowCast v1.0 is a batch-and-near-real-time analytics product for one corridor. It must run reliably on a single workstation, be reproducible end-to-end from raw inputs, and expose every prediction through a dashboard. Multi-city scale-out, live signal control and streaming ingestion are explicitly out of scope for this release.

2. Business Problem

The Northline Corridor carries roughly 210,000 vehicle trips on a weekday. The operations team currently reacts to congestion after it forms: signal plans are fixed by time-of-day, incident detection depends on manual monitoring, and there is no quantitative forecast of how conditions will evolve over the next 30 to 120 minutes. This reactive posture produces measurable costs.

- Recurrent congestion builds faster than fixed signal plans can respond, extending peak periods and lengthening average corridor travel time.
- Weather events (rain reducing visibility, wet-surface braking) degrade flow in ways the current time-of-day plans do not anticipate.
- Incident and accident risk is not quantified ahead of time, so patrol and response resources are positioned reactively rather than predictively.
- Planners have no unified view that ties historical patterns, weather, events and holidays to observed flow, so infrastructure decisions rely on intuition rather than evidence.

MMS needs a system that turns the corridor's existing telemetry into accurate short-horizon forecasts and a clear operational picture. The core business question FlowCast must answer: given everything we know up to now, what will each road segment look like in the next few time windows, and where is risk concentrating?

> **Value hypothesis**
> If the operations team can see congestion, travel time and accident risk 30–120 minutes ahead with quantified confidence, they can pre-empt signal adjustments and reposition response resources — reducing average peak-period travel time and improving incident response times across the corridor.

3. Objectives

3.1 Product objectives

- Forecast traffic volume and average speed per segment for the next 1–4 time windows (30–120 minutes ahead).
- Classify near-term congestion level (Free-flow / Moderate / Heavy / Severe) per segment and time window.
- Estimate corridor and segment travel time from predicted flow and speed.
- Score accident risk per segment and time window as a probability, so response can be prioritised.
- Quantify and display prediction confidence alongside every forecast.
- Surface all outputs, plus historical analytics and model diagnostics, through a single dashboard.

3.2 Engineering objectives (learning outcomes)

Because this build doubles as a final-year engineering capstone, the objectives are also framed as competencies the engineer must demonstrate end-to-end:

- Acquire and merge heterogeneous sources (sensor logs, weather records, calendar/event tables) into one analysis-ready dataset.
- Diagnose and repair real data-quality defects — missing readings, duplicates, unit inconsistencies, sensor-fault outliers — with a documented, reproducible cleaning pipeline.
- Apply the underlying mathematics (linear algebra, probability, statistics, optimisation) explicitly rather than treating libraries as black boxes.
- Build, tune and compare multiple classical ML models on both regression and classification targets.
- Design, implement and train a recurrent deep-learning model from scratch for sequence forecasting, and benchmark it against the classical baselines.
- Establish a defensible evaluation protocol with time-aware validation and the correct metric for each task.
- Package the whole system so that a reviewer can reproduce every result and interact with the predictions.

3.3 Success criteria

Dimension	Target	Measured by
Volume forecast accuracy	MAPE ≤ 12% on the hold-out period	Time-based test split
Congestion classification	Macro-F1 ≥ 0.80 across four classes	Confusion matrix + F1
Accident-risk ranking	ROC-AUC ≥ 0.75	ROC curve on test set
DL vs. ML benchmark	Sequence model beats best classical model on volume RMSE	Head-to-head test RMSE
Reproducibility	Full pipeline reruns from raw inputs with one command	Fresh-environment rerun
Dashboard	All nine required views render on real outputs	Acceptance walkthrough

4. Functional Requirements

Functional requirements are grouped by capability area and carry an identifier for traceability. Priority uses MoSCoW (M = Must, S = Should, C = Could).

4.1 Data acquisition & preparation

ID	Requirement	Priority
FR-01	Ingest raw sensor, weather and calendar tables from the project data directory.	M
FR-02	Validate schema, types and value ranges on load, and log every rejected record.	M
FR-03	Detect and resolve missing readings, duplicates and sensor-fault outliers.	M
FR-04	Merge sources into one segment-time-window table on road ID and timestamp.	M
FR-05	Generate engineered features (lags, rolling stats, time encodings, weather flags).	M
FR-06	Persist a versioned, analysis-ready dataset and a data-quality report.	S

4.2 Modelling & prediction

ID	Requirement	Priority
FR-07	Train Linear Regression, Decision Tree, Random Forest and Gradient Boosting (XGBoost) on the regression targets.	M
FR-08	Train classifiers for congestion level and accident risk, including an SVM baseline.	M
FR-09	Implement and train an LSTM/GRU sequence model from scratch (no pre-trained weights).	M
FR-10	Produce volume, congestion, travel-time and accident-risk predictions per segment/window.	M
FR-11	Attach a confidence/uncertainty estimate to each prediction.	S
FR-12	Persist trained models and reload them for inference without retraining.	M

4.3 Analytics & dashboard

ID	Requirement	Priority
FR-13	Present live/near-term predictions per segment on an interactive dashboard.	M
FR-14	Show historical trends, congestion heatmaps and road-comparison charts.	M
FR-15	Display model-performance metrics, feature importance and forecast visualisations.	M
FR-16	Provide a weather-vs-traffic analysis view.	S
FR-17	Allow the user to upload a new data file and trigger a prediction run.	S
FR-18	Export an insights/report summary of a selected time range.	C

5. Non-Functional Requirements

Category	Requirement	Target
Performance	Batch inference over the full corridor for one horizon completes quickly on a workstation.	≤ 30 s
Scalability	Pipeline handles the full multi-month dataset in memory on a 16 GB machine.	≥ 150k rows
Reproducibility	All randomness is seeded; a clean rerun reproduces reported metrics.	Deterministic
Maintainability	Code is modular (ingest / clean / features / model / serve) and documented.	PEP 8 + docstrings
Usability	Dashboard is navigable by a non-technical operator without training.	≤ 3 clicks to any view
Reliability	Bad input rows are quarantined, not silently dropped; failures are logged.	0 silent drops
Portability	Runs on Windows, macOS or Linux via a pinned environment file.	1 setup command
Transparency	Every prediction is traceable to its inputs and model version.	Full lineage

6. User Roles

FlowCast serves three primary roles inside the corridor operations centre. The engineer building the system must design the dashboard and outputs around what each role needs to do.

Role	Goal	Key interactions
Traffic Operations Analyst	Anticipate congestion and act before it forms.	Reads live forecasts, heatmaps and confidence; flags segments for signal review.
Incident Response Coordinator	Position patrols where risk is rising.	Monitors accident-risk scores and ranked segment list.
Transport Planner	Make evidence-based infrastructure decisions.	Explores historical trends, weather impact and road-comparison analytics.
System Owner / Reviewer	Trust and audit the model.	Inspects performance metrics, feature importance and data-quality reports.

7. System Modules

FlowCast is decomposed into seven modules with clear inputs and outputs. Each module is independently testable, which lets the engineer build and validate the pipeline incrementally.

#	Module	Responsibility
M1	Ingestion & Validation	Loads raw sensor, weather and calendar tables; enforces schema and range checks; quarantines invalid rows.

#	Module	Responsibility
M2	Cleaning & Wrangling	Handles missing values, duplicates, outliers, unit and category harmonisation; merges sources into one table.
M3	Feature Engineering	Builds lag features, rolling statistics, time-window encodings, weather flags and interaction terms.
M4	EDA & Reporting	Produces distributions, correlations and the data-quality report that justifies later modelling choices.
M5	Classical ML Engine	Trains, tunes and compares regression and classification models; exports the winners.
M6	Deep-Learning Engine	Builds and trains the recurrent sequence model from scratch; benchmarks against M5.
M7	Analytics Dashboard	Serves predictions, historical analytics, model diagnostics and reports to end users.

Module data flow

`raw tables` → `M1 ingest/validate` → `M2 clean/merge` → `M3 features` → `M4 EDA` → `M5 / M6 models` → `M7 dashboard`

8. Dataset Specification

The corridor dataset is delivered as three linked source tables that the engineer must integrate. Together they cover 25 road segments over a continuous multi-month period at 30-minute aggregation, yielding on the order of 150,000–180,000 segment-time-window records once merged. The tables are intentionally raw: they carry the gaps, duplicates and inconsistencies typical of production sensor telemetry, and must be cleaned before modelling.

8.1 Source tables

Table	Grain	Contents
traffic_sensor_log	segment × 30-min window	Counts, speed, occupancy and derived flow from loop/radar sensors.
weather_observations	station × hour	Condition, temperature, rainfall, visibility from corridor weather stations.
calendar_events	date	Public-holiday flags, local-event flags and scheduled roadworks.

8.2 Merged schema (analysis-ready record)

Field	Type	Description
road_id	string	Unique segment identifier (e.g. NL-014).
road_name	string	Human-readable segment name.
latitude / longitude	float	Segment mid-point GPS coordinates.
weather_station_id	string	Join key linking each segment reading to its weather station; dropped after the merge.
date	date	Calendar date of the observation.

Field	Type	Description
time	time	Start of the 30-minute window.
day_of_week	categorical	Mon–Sun, derived from date.
traffic_volume	int	Vehicles observed in the window (primary regression target).
vehicle_count	int	Raw sensor count before flow adjustment.
vehicle_type_dist	json/obj	Share of two-wheeler / car / LCV / HCV.
avg_speed	float	Mean speed in the window (km/h).
occupancy	float	Detector occupancy 0–100%; supports congestion derivation and feature engineering.
congestion_level	categorical	Free-flow / Moderate / Heavy / Severe (classification target).
travel_time	float	Segment traversal time (minutes).
weather_condition	categorical	Clear / Cloudy / Rain / Fog etc.
temperature	float	Air temperature (°C).
rainfall	float	Rainfall in the window (mm).
visibility	float	Visibility (metres).
public_holiday	bool	1 if the date is a public holiday.
event_flag	bool	1 if a local event affects the segment.
accident_count	int	Reported incidents in the window (accident-risk target basis).
signal_timing	int	Effective green time (seconds) for the segment.
road_capacity	int	Design capacity (vehicles/hour) for the segment.

8.3 Known data-quality issues to expect

These defects are present in the source tables and must be handled by the cleaning pipeline (Section 9). They are the reason the dataset requires substantial preparation before it can be modelled.

- Missing sensor windows where a detector dropped out, leaving null volume/speed for stretches of time.
- Duplicate log rows from sensor retries — same segment and timestamp recorded more than once.
- Physically impossible outliers: negative counts, speeds above 200 km/h, occupancy over 100% from faulty detectors.
- Weather recorded hourly against 30-minute traffic windows, requiring temporal alignment and forward/interpolated fill.
- Inconsistent categorical labels for weather ('Rain', 'rainy', 'RAIN') that must be normalised.
- Mixed date formats across source files — weather uses DD/MM/YYYY while traffic uses YYYY-MM-DD — requiring both to be parsed to a common timestamp before joining.
- Congestion level missing for some rows, requiring derivation from the volume-to-capacity ratio.

Companion reference

A separate FlowCast Data Dictionary accompanies the raw files. It documents every column in the three source tables, the exact join keys between them, the congestion-derivation rule and the full list of data-quality issues to resolve. Keep it open alongside the data while building the cleaning pipeline.

9. Data Cleaning Pipeline

The cleaning pipeline is a deterministic, ordered sequence of Pandas/NumPy transformations. Order matters: validation precedes imputation, deduplication precedes merging, and scaling happens only after the train/test split so that statistics do not leak from test into train. Every stage writes a log line so the data-quality report can quantify what changed.

9.1 Ordered stages

1. Schema & type validation — enforce column types; parse mixed datetime formats into a single tz-aware timestamp; range-check numeric fields.
2. Duplicate removal — drop exact and key-level (road_id + timestamp) duplicates, keeping the most complete record.
3. Outlier detection — flag physically impossible values by rule, and statistically extreme values by z-score / IQR; cap or remove per field policy.
4. Missing-value handling — impute short sensor gaps by time interpolation, longer gaps by segment-and-window medians; drop rows that cannot be safely recovered.
5. Categorical harmonisation — normalise weather and day labels to a controlled vocabulary; encode ordinal (congestion) and nominal (weather) variables appropriately.
6. Temporal alignment & merge — align hourly weather to 30-minute traffic windows; left-join weather and calendar onto the sensor grain on road_id and timestamp.
7. Feature engineering — derive time-window features, lag and rolling-window statistics, volume-to-capacity ratio, rainfall/visibility flags and holiday/event interactions.
8. Data validation & quality checks — assert no nulls in modelling columns, verify row counts, confirm value ranges; emit the final data-quality report.

9.2 Feature engineering detail

Feature group	Examples
Temporal	Hour-of-day and day-of-week cyclical encodings (sin/cos), weekend flag, peak-period flag.
Lag features	Volume and speed at t-1, t-2, t-48 (same window previous day) per segment.
Rolling statistics	Rolling mean/std of volume and speed over the previous 4 and 8 windows.
Capacity	Volume-to-capacity ratio; headroom to road_capacity.
Weather	Rain flag, low-visibility flag, temperature bands.
Calendar	Holiday × peak interaction; event proximity.

Leakage guard

Lag and rolling features are computed within each segment in strict time order, and all scaling parameters are fit on the training window only. No feature may use information from a future window relative to the prediction time.

10. Mathematics Used

FlowCast is deliberately structured so that each required mathematical concept has a concrete home in the pipeline. The engineer must be able to explain not just how a library call works, but the mathematics underneath it. The table maps every concept to where it is applied.

10.1 Concept-to-application map

Mathematical concept	Where it is applied in FlowCast
Linear algebra, matrices, vectors	The cleaned dataset is a feature matrix X (rows = records, columns = features); each record is a feature vector. Model training operates on matrix–vector products.
Matrix operations / dot product	Linear-regression prediction is Xw; a neural layer computes Wx + b. Both are implemented as matrix multiplications.
Probability	Accident-risk output is a probability; classifier outputs are class probabilities used for thresholding and ranking.
Descriptive statistics (mean, median, mode)	EDA summaries; median-based imputation of sensor gaps; mode for categorical fills.
Standard deviation & variance	Rolling-window volatility features; spread analysis in EDA; basis for z-score scaling.
Correlation	Feature–target and feature–feature correlation to guide feature selection and spot redundancy.
Covariance	Covariance structure among features underpins correlation and multicollinearity checks.
Normal distribution	Residual analysis for regression; z-score outlier detection assumes approximate normality of well-behaved sensor readings.
Feature scaling	Required before SVM and the neural network so no feature dominates by magnitude.
Min–Max normalisation	Scales bounded features (e.g. visibility, occupancy) into [0,1] for the deep-learning model.
Z-score standardisation	Standardises unbounded features (volume, speed) to mean 0, variance 1 for gradient-based models.
Gradient descent	Used to train linear regression from first principles and to optimise the neural network via backpropagation.
Cost / loss functions	MSE/RMSE for regression targets; cross-entropy for congestion and accident-risk classification.

10.2 Evaluation mathematics

Metric	Definition & use in FlowCast
Confusion matrix	Counts of true/false positives and negatives per class — the basis for all classification metrics on congestion and accident risk.
Precision	Of the windows flagged as a congestion/risk class, the fraction that truly were — controls false alarms to operators.

Metric	Definition & use in FlowCast
Recall	Of the windows that truly were a class, the fraction caught — controls missed congestion or missed risk.
F1 score	Harmonic mean of precision and recall; macro-F1 is the headline classification metric across the four congestion classes.
ROC-AUC	Ranking quality of the accident-risk probability across all thresholds; the headline risk metric.

Build-it-once rule

At least one linear-regression training loop and the loss/gradient computations must be implemented directly with NumPy before the engineer relies on library implementations. This demonstrates the gradient-descent and cost-function mathematics rather than hiding them behind a call.

11. Machine Learning Workflow

The classical ML engine (M5) establishes strong, interpretable baselines before the deep-learning model is attempted. It covers both prediction types FlowCast needs: regression for continuous targets (traffic volume, travel time) and classification for categorical targets (congestion level, accident risk).

11.1 Targets and framing

Target	Task type	Framing
Traffic volume	Regression	Predict next-window volume per segment from lag, weather and calendar features.
Travel time	Regression	Predict segment traversal time from predicted flow, speed and capacity.
Congestion level	Classification (4-class)	Predict Free-flow / Moderate / Heavy / Severe for the next window.
Accident risk	Classification (binary)	Predict probability of an incident in the next window per segment.

11.2 Models to build and compare

- Linear Regression — interpretable baseline; also implemented once from scratch with NumPy gradient descent to ground the mathematics.
- Decision Tree — captures non-linear thresholds; used to read off simple, explainable rules.
- Random Forest — bagged ensemble for variance reduction and a first feature-importance ranking.
- Gradient Boosting (XGBoost) — expected top classical performer on tabular targets; carefully tuned.
- Support Vector Machine — margin-based baseline for the classification targets on scaled features.

11.3 Workflow steps

9. Split the data by time — earliest windows for training, latest for testing — never randomly, to respect temporal ordering.
10. Fit scalers on the training split only; apply to validation and test.
11. Train each model with cross-validated hyperparameter search on the training window (time-series CV).

12. Evaluate every model on the untouched test window using the task-appropriate metrics.
13. Compare models on a single scoreboard; inspect feature importance and error patterns.
14. Select and justify the winning classical model per target; export it for the dashboard.

11.4 Model-selection justification

Selection is not by headline accuracy alone. The engineer must weigh the metric that matters for each task (e.g. recall on accident risk, macro-F1 on congestion), the stability of the score across CV folds, inference cost, and interpretability for the operations team. The final write-up must state why the chosen model beats the alternatives on the criteria that matter operationally, not merely on a single number.

12. Deep Learning Workflow

Traffic is a temporal process: the next window depends on the recent trajectory of flow, speed and conditions. The deep-learning engine (M6) models this sequence structure with a recurrent network built and trained from scratch — no pre-trained weights and no transfer learning. Its purpose is both to (potentially) beat the classical baselines on volume forecasting and to demonstrate the engineer's ability to design and train a network end-to-end.

12.1 Model choice

The primary model is an LSTM (or GRU) sequence forecaster. A Bidirectional LSTM variant is trained for comparison on the historical-analysis view where full-window context is available. A Temporal Convolutional Network is an optional stretch comparison.

Component	Design
Input	A sliding window of the previous N time steps per segment, each step a vector of scaled features.
Core	One or two LSTM/GRU layers with dropout between them; hidden size tuned by search.
Head	A dense layer mapping the final hidden state to the forecast (volume regression; a second head for congestion class).
Output	Next-window volume (and, for the multi-task variant, congestion probabilities).

12.2 Training procedure

15. Build sequences: for each segment, slide an N-step window forward to form (sequence → next-window target) pairs, preserving time order.
16. Split sequences by time into train / validation / test, mirroring the classical split.
17. Define the loss (MSE for volume, cross-entropy for the classification head) and an Adam optimiser.
18. Train with mini-batches; monitor validation loss each epoch.
19. Tune hyperparameters — sequence length N, hidden size, layer count, dropout rate, learning rate, batch size.
20. Apply overfitting controls: dropout, early stopping on validation loss, and weight regularisation as needed.
21. Evaluate on the held-out test window with the same metrics as the classical models.

12.3 Overfitting prevention

- Dropout between recurrent layers and before the dense head.

- Early stopping keyed to validation loss, restoring best weights.
- Learning-rate reduction on plateau.
- Train/validation loss curves inspected every run to confirm convergence rather than memorisation.

12.4 Benchmark against classical models

> **Head-to-head requirement**
> The sequence model must be evaluated on the exact same test window and metrics as the classical models. The final report presents a single comparison table (RMSE/MAPE for volume, F1/AUC for classification) and states plainly whether the deep model earned its added complexity.

13. Model Evaluation Strategy

13.1 Validation design

Because traffic is a time series, evaluation uses time-based splitting and time-series cross-validation, never random k-fold. The earliest portion of the timeline trains the models, a middle slice validates and tunes them, and the latest, fully unseen slice tests them. This mirrors how FlowCast will be used in production: predicting the future from the past.

13.2 Metrics by task

Task	Primary metric	Supporting metrics
Volume / travel-time regression	RMSE	MAE, MAPE, R^2
Congestion classification (4-class)	Macro-F1	Precision, recall, confusion matrix, accuracy
Accident-risk classification	ROC-AUC	Precision, recall, F1 at operating threshold

13.3 Error analysis

- Break errors down by time-of-day, day-of-week and weather condition to reveal where each model struggles.
- Inspect the confusion matrix to see which congestion classes are confused (e.g. Heavy vs Severe).
- Plot predicted vs actual volume over the test timeline for representative segments.
- Review residual distributions for bias and heteroscedasticity.

13.4 Model governance

Each trained model is versioned with its training window, feature set, hyperparameters and test metrics recorded in a model card, so every prediction shown on the dashboard is traceable to a specific, reproducible model.

14. Dashboard Design

The dashboard (M7) is the product surface. It is organised into views that map to the three operational roles. It reads persisted model outputs and analytics tables; it does not retrain on the fly. Navigation is flat — every view is reachable in a few clicks.

14.1 Required views

View	What it shows
Live prediction	Next-window volume, congestion level, travel time and accident risk per segment, with confidence.
Historical trends	Volume and speed over time per segment, with peak-hour patterns.
Congestion heatmap	Segment × time-window grid coloured by congestion severity.
Road comparison	Side-by-side charts comparing segments on volume, speed and reliability.
Model performance	Test metrics per model and target, with the classical-vs-deep comparison.
Feature importance	Ranked drivers of the predictions from the tree ensembles.
Forecast visualisation	Predicted-vs-actual overlays across the forecast horizon.
Prediction confidence	Uncertainty bands around each forecast.
Weather vs traffic	How rainfall, visibility and condition relate to observed and predicted flow.

14.2 Supporting modules

- Data-upload module — accept a new data file, run validation and cleaning, and refresh predictions.
- Model-training interface — trigger a retraining run and view the resulting metrics.
- Prediction interface — request a forecast for a chosen segment and horizon.
- Reports & insights — export a summary of a selected time range and its key findings.

14.3 Design principles

- Every number on screen is a real model output or a real analytics aggregate — no placeholder figures.
- Forecasts always appear with their confidence so operators can calibrate trust.
- Colour encodes severity consistently across every view (free-flow → severe).
- The heatmap and comparison views are the default landing surfaces for analysts and planners respectively.

15. Technology Stack

The stack is intentionally standard and open-source so results are reproducible on any workstation. Versions are pinned in an environment file.

Layer	Technology	Role
Language	Python 3.11	Primary implementation language.
Numerics	NumPy	Matrix/vector operations; from-scratch gradient descent.
Data	Pandas	Cleaning, wrangling, merging, feature engineering.
Classical ML	scikit-learn, XGBoost	Regression, classification, model selection, metrics.
Deep learning	TensorFlow/Keras or PyTorch	LSTM/GRU sequence model built and trained from scratch.

Layer	Technology	Role
Visualisation	Matplotlib, Seaborn, Plotly	EDA charts and dashboard figures.
Dashboard	Streamlit	Interactive analytics and prediction UI.
Notebooks	Jupyter	EDA and experiment narrative.
Versioning	Git	Source control and experiment history.
Environment	conda / venv + requirements.txt	Pinned, reproducible setup.

16. Folder Structure

A conventional, modular layout keeps ingestion, cleaning, features, modelling and serving separable and testable.

```
flowcast/
├── data/
│   ├── raw/                # source tables as delivered (read-only)
│   ├── interim/            # cleaned, pre-merge intermediates
│   └── processed/          # analysis-ready, versioned dataset
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_cleaning.ipynb
│   └── 03_modelling.ipynb
├── src/
│   ├── ingest.py           # M1 load + validate
│   ├── clean.py            # M2 clean + merge
│   ├── features.py         # M3 feature engineering
│   ├── ml_models.py        # M5 classical models
│   ├── dl_model.py         # M6 LSTM/GRU from scratch
│   ├── evaluate.py         # metrics + comparison
│   └── utils.py
├── models/                 # persisted models + model cards
├── reports/
│   ├── data_quality.md
│   └── figures/
├── dashboard/
│   └── app.py              # M7 Streamlit app
├── tests/
├── requirements.txt
├── config.yaml
└── README.md
```

17. Development Phases

The build is delivered by one engineer across four one-week phases. Each week ends with a concrete, reviewable artefact, so slippage is caught early and the pipeline is always in a demonstrable state.

Week 1 — Data engineering & EDA

- Stand up the repo, environment and config; load the three source tables (M1).
- Build the full cleaning pipeline: validation, dedup, outliers, imputation, harmonisation, merge (M2).
- Engineer the feature set and produce the analysis-ready dataset (M3).
- Run EDA and publish the data-quality report (M4).

Exit artefact: versioned processed dataset + data-quality report + EDA notebook.

Week 2 — Classical machine learning

- Implement time-based splitting and the from-scratch linear-regression baseline.
- Train and tune Linear Regression, Decision Tree, Random Forest, XGBoost and SVM (M5).
- Evaluate all models on regression and classification targets; assemble the scoreboard.
- Select and justify the winning classical model per target; export models + model cards.

Exit artefact: trained classical models + comparison table + selection write-up.

Week 3 — Deep learning

- Build sequence windows; implement the LSTM/GRU model from scratch (M6).
- Train, tune hyperparameters and apply overfitting controls; log loss curves.
- Evaluate on the same test window and metrics; run the head-to-head benchmark vs classical models.
- Persist the deep model and its model card.

Exit artefact: trained sequence model + classical-vs-deep benchmark.

Week 4 — Dashboard, reporting & packaging

- Build the Streamlit dashboard with all nine required views on real outputs (M7).
- Add the data-upload, prediction and reporting modules.
- Write the README, reproduce the full pipeline end-to-end from raw data, and finalise reports.
- Acceptance walkthrough against the success criteria in Section 3.3.

Exit artefact: running dashboard + reproducible pipeline + final report.

17.1 Milestone summary

Week	Focus	Reviewable output
1	Data engineering & EDA	Processed dataset + data-quality report
2	Classical ML	Model scoreboard + selection justification
3	Deep learning	Sequence model + benchmark table
4	Dashboard & delivery	Live dashboard + reproducible pipeline + report

18. Expected Deliverables

22. Reproducible source repository with the modular layout in Section 16.
23. Analysis-ready processed dataset plus the raw-to-processed cleaning pipeline.
24. Data-quality report documenting every defect found and how it was resolved.
25. EDA notebook with distributions, correlations and the insights that drove modelling choices.
26. Trained classical models (regression + classification) with a comparison scoreboard.
27. Deep-learning sequence model trained from scratch, with training curves and the classical-vs-deep benchmark.
28. Model cards recording training window, features, hyperparameters and test metrics for every model.
29. Interactive Streamlit dashboard covering all nine required views plus upload/predict/report modules.
30. Final technical report tying together the mathematics, methods, results and recommendations.

31. README with one-command setup and end-to-end reproduction instructions.

19. Future Enhancements

Out of scope for v1.0 but recorded to guide the roadmap once the single-corridor build is proven:

- Real-time streaming ingestion to replace batch refreshes.
- Multi-corridor and multi-city scale-out with per-network models.
- Graph-based models that exploit the road-network topology between segments.
- Attention/transformer sequence models benchmarked against the recurrent baseline.
- Closed-loop integration that recommends signal-timing changes to the control system.
- Probabilistic forecasting with calibrated prediction intervals throughout.
- Automated retraining and drift monitoring in production.

Tools Required

Everything needed to complete FlowCast on a single machine within the four-week sprint. All software is free and open-source; no paid services are required.

Hardware

Item	Recommendation
Machine	Laptop/desktop with an 8-core CPU and 16 GB RAM (8 GB workable for reduced sequence lengths).
Storage	~10 GB free for the dataset, intermediates and trained models.
GPU	Optional. A CUDA GPU speeds up deep-learning training but is not required; the model trains on CPU within the timeline.

Core software & libraries

Category	Tools
Language & runtime	Python 3.11
Numerics & data	NumPy, Pandas
Classical ML	scikit-learn, XGBoost
Deep learning	TensorFlow/Keras or PyTorch
Visualisation	Matplotlib, Seaborn, Plotly
Dashboard	Streamlit
Notebooks & IDE	Jupyter, VS Code (or PyCharm)
Environment mgmt	conda or venv + requirements.txt
Version control	Git (GitHub/GitLab for hosting)
Utilities	PyYAML (config), joblib/pickle (model persistence), tqdm (progress)

Skills the engineer must bring

- Python and the NumPy/Pandas data stack.
- Statistics and the linear-algebra / calculus behind gradient descent and loss functions.
- Machine-learning modelling, tuning and honest evaluation.
- Deep-learning fundamentals: designing, training and regularising a recurrent network.
- Basic software engineering: modular code, version control, reproducible environments.

Definition of done

FlowCast v1.0 is complete when a reviewer can clone the repository, run one setup command, reproduce every reported metric from the raw data, and open a dashboard where all nine views display real predictions and analytics for the Northline Corridor — all achieved by one engineer within four weeks.